

Contents

guide LoRA: Low-Rank Adaptation of Large Language Models	1
0. 这篇论文到底在改写什么	1
1. 回到 2021 年的语境	2
2. 原论文结构地图	2
3. 方法图: LoRA 在一层 Linear 旁边做什么	3
4. 张量级别拆解	3
5. 为什么低秩假设有道理	4
6. 初始化: 为什么 A 随机、B 为零	4
7. alpha/r scaling 是什么	5
8. LoRA 应该加在哪些矩阵上	5
9. 和 Adapter/Prefix 的工程差异	5
10. 最小代码实现	6
11. 实验证据链	6
12. Rank analysis: r 为什么可以很小	7
13. LoRA 的局限	7
14. 和本仓库代码怎么对上	7
15. 30 分钟本地实验	8
16. 今天为什么还要读	8
17. 常见误区	8
18. 闭卷掌握检查	9
19. 用 AI agent 学这篇的正确方式	9

guide_LoRA: Low-Rank Adaptation of Large Language Models

原论文: LoRA: Low-Rank Adaptation of Large Language Models

本地原文 PDF: [learning/lora-family/paper/01_lora.pdf](#)

作者: Hu et al.

年份: 2021

类型: paper

0. 这篇论文到底在改写什么

LoRA 改写的是大模型微调的成本结构。传统 full fine-tuning 会更新所有参数；模型越大，每个任务保存一份完整模型越不现实。LoRA 的主张是：预训练权重 W_0 不动，下游任务只学习一个低秩增量 $\Delta W = BA$ 。训练时走并行低秩分支，部署时把 BA 合并回 W_0 ，因此没有 adapter 那种额外推理深度。

论文摘要里给了很醒目的量级：对 GPT-3 175B，用 Adam full fine-tuning 时，LoRA 可以把可训练参数减少约 10,000 倍，把 GPU memory requirement 降低约 3 倍；在 RoBERTa、DeBERTa、GPT-2、GPT-3 上质量接近或超过 full fine-tuning，同时训练吞吐更高，且没有额外 inference latency。

这篇是 PEFT 里最重要的母论文之一。后面的 QLoRA、AdaLoRA、PiSSA、LoftQ、DoRA、rsLoRA 都是在问：如果 $\Delta W = BA$ 这条路成立，我们还能不能让它更省显存、更好初始化、更稳定、更接近全参微调？

1. 回到 2021 年的语境

在 LoRA 之前，大模型 adaptation 大致有几条路：

1. **Full fine-tuning**: 效果强，但每个任务都要保存完整参数副本，并且 Adam optimizer states 和 gradients 让显存压力巨大。
2. **Adapter layers**: 在 Transformer block 里插入小 MLP，参数少，但推理多了顺序执行层，低 batch online serving 下会增加 latency。
3. **Prefix/prompt tuning**: 学连续 prompt 或每层 prefix activations，参数少，但训练可能不稳定，而且会占用可用 sequence length。

LoRA 的目标很具体：

- 参数要少。
- 训练显存要少。
- 多任务切换要便宜。
- 部署不能增加推理延迟。
- 效果不能明显低于 full fine-tuning。

这几个目标放在一起很难。Adapter 省参数但增延迟；prefix 省参数但占上下文；full fine-tuning 效果好但太贵。LoRA 的设计点是：不在网络深度上加模块，而是在已有线性层权重旁边加一个可合并的低秩增量。

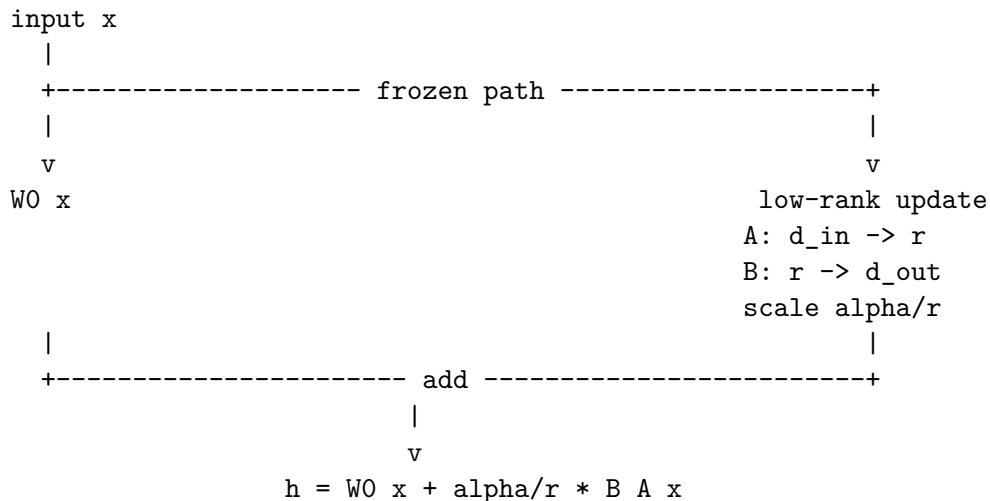
2. 原论文结构地图

建议按下面顺序读原文：

1. Abstract/Introduction: 看 GPT-3 175B 的部署和训练成本问题。
2. Section 2/3: 看 full fine-tuning、adapter、prefix tuning 的 trade-off。
3. Section 4.1: LoRA 的核心公式 $W_0 + BA$ 。
4. Figure 1: LoRA 在已有 weight matrix 旁边加低秩分支。
5. Section 4.2: Transformer 里到底给哪些矩阵加 LoRA。
6. Section 4.3: practical benefits and limitations，尤其是显存、checkpoint、任务切换。
7. Experiments: RoBERTa/DeBERTa/GPT-2/GPT-3 上的质量和参数对比。
8. Rank analysis: 看为什么很小的 rank 也可能够用。

如果你只读一次，必须读懂 Section 4.1 和 Figure 1。LoRA 的所有后续变体基本都在改这一张图里的某个元素：rank、初始化、scaling、量化、矩阵分解形式或 merge 方式。

3. 方法图: LoRA 在一层 Linear 旁边做什么



注意 LoRA 分支和原线性层是并行关系，不是像 adapter 那样串在后面。因此训练时多了一条低秩支路，部署时可以直接把 $\alpha/r * B A$ 加到 W_0 里：

$$W_{merged} = W_0 + \alpha/r * B A$$

合并后模型结构和 full fine-tuned linear layer 完全一样，所以推理 latency 不增加。

4. 张量级别拆解

原论文符号里，预训练权重：

W_0 : $[d_{out}, d_{in}]$ frozen

LoRA 学两个低秩矩阵：

A: $[r, d_{in}]$ trainable

B: $[d_{out}, r]$ trainable

BA: $[d_{out}, d_{in}]$

对输入：

x: $[batch, seq, d_{in}]$

base: $x @ W_0.T \rightarrow [batch, seq, d_{out}]$

lora: $x @ A.T @ B.T \rightarrow [batch, seq, d_{out}]$

output: $base + \alpha/r * lora \rightarrow [batch, seq, d_{out}]$

参数量从：

full fine-tune one matrix: $d_{out} * d_{in}$

LoRA one matrix: $r * d_{in} + d_{out} * r$

如果 $d_{in} = d_{out} = d$ ，就是：

full: d^2

LoRA: $2 r d$

举例 $d=768$, $r=8$:

full: $768 * 768 = 589,824$
LoRA: $2 * 8 * 768 = 12,288$
ratio: about 48x fewer trainable parameters for this matrix

本仓库 LoRAGPT2 默认给 GPT-2 的 `c_attn` 打 LoRA。`c_attn` 是合并的 q/k/v projection, $d_{out} = 3 * d_{model} = 2304$, 所以每层:

A: $[8, 768] = 6,144$
B: $[2304, 8] = 18,432$
total per layer = 24,576
12 layers = 294,912 trainable params

这和 `learning/lora-family/src/lora_minimal.py` 里的注释一致。

5. 为什么低秩假设有道理

LoRA 的理论 motivation 来自 intrinsic dimension 这类观察: 预训练大模型虽然参数很多, 但下游任务 adaptation 需要移动的有效自由度可能很低。

Full fine-tuning 学的是:

$W_{ft} = W_0 + \Delta W$

LoRA 假设 ΔW 不需要 full rank:

$\Delta W \approx B A, \text{rank}(\Delta W) \leq r$

这不是数学定理, 而是经验假设。论文后面用 rank ablation 和对 full fine-tuning update 的分析支持它。直觉上, 预训练模型已经学到大量通用表示, 下游任务只是把行为推向某些方向; 这些方向可能集中在一个低维子空间里。

这也是 LoRA 和 adapter 的本质差异:

- adapter 说: 我在 activation path 上加一个小模块。
- LoRA 说: 我直接约束 weight update 的秩。

6. 初始化: 为什么 A 随机、B 为零

论文设置:

$A \sim \text{random Gaussian} / \text{Kaiming}$
 $B = 0$

这样训练开始时:

$\Delta W = B A = 0$

也就是说, 模型初始行为完全等同于预训练模型, 不会突然扰动 W_0 。

为什么不能两个都为零? 因为那样两个分支都没有有效梯度方向。为什么常见实现是 A 随机、B 零? 因为第一步 backward 时, B 可以收到非零梯度, 训练立刻开始; 同时前向输出仍然不变。

最小代码:

```
self.A = nn.Parameter(torch.empty(r, d_in))
self.B = nn.Parameter(torch.zeros(d_out, r))
nn.init.kaiming_uniform_(self.A, a=math.sqrt(5))
```

这正是本仓库 LoRALinear 的实现。

7. alpha/r scaling 是什么

LoRA 前向不是简单加 BAx ，而是：

$$h = W_0 x + (\alpha / r) * B A x$$

α/r 是更新幅度的缩放。它让 rank 改变时，LoRA 分支的整体尺度更容易保持可控。你可以把 α 理解成 LoRA 分支的“学习强度旋钮”。

实践中：

- r 控制容量。
- α 控制更新尺度。
- α/r 影响训练稳定和最终增量大小。

很多新手只调 r ，忽略 α 。这会导致一个错觉：rank 变了之后效果变化不一定来自容量，也可能来自更新尺度变化。

8. LoRA 应该加在哪些矩阵上

Transformer attention 里常见线性矩阵：

W_q, W_k, W_v, W_o

MLP 里还有 up/down projection。原论文说原则上 LoRA 可以加到任意 dense layer。实验里为了简单，很多设置主要加在 W_q 和 W_v ，后来工程实践常根据模型结构选择 q/k/v/o 或 MLP。

为什么 attention projection 很自然？

- attention 负责 token 间信息路由。
- 下游任务往往需要改变“看哪里”和“取什么信息”。
- q/v projection 的低秩调整常能有效改变行为。

但这不是铁律。现代 PEFT 经常对 q_proj/k_proj/v_proj/o_proj/gate_proj/up_proj/down_proj 做组合搜索。

9. 和 Adapter/Prefix 的工程差异

原论文对 adapter latency 很在意。Adapter 虽然参数少，但它在网络里增加额外层。对于 online inference，尤其 batch size 小、模型并行多卡时，额外 sequential operations 会带来真实 latency。

Prefix tuning 的问题不同：它把可训练信息放进序列或每层 KV 前缀里，可能：

- 占用上下文长度。
- 训练不稳定。
- 性能随可训练参数数量非单调变化。

LoRA 的工程优势：

training：

- only A/B require grad
- optimizer states only for A/B
- lower VRAM

deployment：

- merge BA into W_0
- no extra layer

no shorter context

```
multi-task:
    one base model
    many small LoRA modules
```

论文给 GPT-3 175B 的估算很直观: full fine-tuning 训练显存约 1.2TB, LoRA 可降到约 350GB; 如果 $r=4$ 且只适配 query/value projection, checkpoint 从约 350GB 降到约 35MB, 约 10,000x。

10. 最小代码实现

下面是论文机制的最小实现:

```
class LoRALinear(nn.Module):
    def __init__(self, base, r=8, alpha=16):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False

        d_in = base.in_features
        d_out = base.out_features
        self.A = nn.Parameter(torch.empty(r, d_in))
        self.B = nn.Parameter(torch.zeros(d_out, r))
        nn.init.kaiming_uniform_(self.A, a=math.sqrt(5))
        self.scaling = alpha / r

    def forward(self, x):
        base = self.base(x)
        lora = x @ self.A.T @ self.B.T
        return base + self.scaling * lora

    def merge(self):
        delta = self.scaling * (self.B @ self.A)
        self.base.weight.data += delta
```

这段代码对应论文三个关键点:

1. base 冻结。
2. A/B 是唯一训练参数。
3. $B @ A$ 可以 merge 回 base weight。

11. 实验证据链

原论文的实验覆盖:

- RoBERTa / DeBERTa: GLUE benchmark。
- GPT-2: E2E NLG Challenge。
- GPT-3 175B: WikiSQL、SAMSum 等生成任务。

主结论:

- LoRA 用更少可训练参数达到或超过强 baselines。

- 相比 adapter, LoRA 没有额外 inference latency。
- 相比 full fine-tuning, LoRA 显著降低训练显存和 checkpoint 存储。
- 很低的 rank 在一些大模型场景下已经够用。

读实验时不要只看 “LoRA 分数高”。要看四条证据是否同时成立:

1. 质量接近 full fine-tuning。
2. 参数量明显少。
3. 训练显存和吞吐更好。
4. 部署不增加延迟。

这四条同时成立, 才是 LoRA 的完整价值。

12. Rank analysis: r 为什么可以很小

论文关心 weight update 的 rank-deficiency。直觉是, 如果 full fine-tuning 的 ΔW 在下游任务中主要集中在少数方向, 那么 $r=1,2,4,8$ 这样的低秩分解就能捕捉大部分有效变化。

但不要把它误解成 “rank 越小越好”。更准确的说法是:

- 小 rank 是强正则, 省参数, 但容量有限。
- 大 rank 更接近 full fine-tuning, 但成本更高, 也可能过拟合。
- 不同任务、模型、层和 target modules 的最佳 rank 不同。

这也是 AdaLoRA 后来要动态分配 rank 的原因: 每层、每矩阵需要的 rank 可能不一样。

13. LoRA 的局限

LoRA 不是免费午餐:

- 如果任务需要大幅改变模型行为, 低 rank 可能不够。
- 多任务同 batch 且每个样本用不同 LoRA 时, merge 后的零延迟优势不好直接保留。
- 选择 target modules、rank、alpha、dropout 仍然需要调。
- LoRA 主要节省训练参数和 optimizer states, 不一定让 forward FLOPs 大幅下降。
- 在量化 base model 上训练 LoRA 会引入新的数值问题, QLoRA 才专门处理这个场景。

理解这些局限, 才能避免把 LoRA 当成任何任务都无脑套的插件。

14. 和本仓库代码怎么对上

优先打开:

- `learning/lora-family/lectures/01-lora.md`
- `learning/lora-family/src/lora_minimal.py`
- `learning/lora-family/src/lora_peft.py`
- `learning/lora-family/src/tests/test_lora_consistency.py` 如果存在同类测试, 可重点看 merge 前后是否一致。

本仓库 LoRALinear 正好对应论文公式:

```
base_out = self.base(x)
lora_out = self.dropout(x) @ self.A.T @ self.B.T
return base_out + self.scaling * lora_out
```

`merge_weights()` 对应部署时:

```
delta = alpha/r * (B @ A)
W0 <- W0 + delta
```

你读代码时要检查:

1. base 参数是否 requires_grad=False。
2. A/B shape 是否和目标 Linear/Conv1D 对齐。
3. B 是否零初始化。
4. scaling 是否是 alpha/r。
5. merge 后 forward 是否与未 merge 时数值一致。

15. 30 分钟本地实验

实验 1: 看 rank 对参数量的影响。

```
def lora_params(d_in, d_out, r):
    return r * d_in + d_out * r

for r in [1, 2, 4, 8, 16, 64]:
    print(r, lora_params(768, 2304, r))
```

实验 2: 验证初始 LoRA 不改变 base 输出。

```
base = nn.Linear(16, 16)
lora = LoRALinear(base, r=4, alpha=8)
x = torch.randn(2, 3, 16)
assert torch.allclose(lora(x), base(x), atol=1e-6)
```

实验 3: 验证 merge 不改变输出。

```
y_before = lora(x)
lora.merge_weights()
y_after = lora.base(x)
assert torch.allclose(y_before, y_after, atol=1e-5)
```

这三个实验分别对应论文的三个 claim: 省参数、初始不扰动、部署无额外延迟。

16. 今天为什么还要读

LoRA 今天仍然是 PEFT 默认语言。你在微调 Llama、Qwen、Mistral、DeepSeek、VLM、diffusion model 时，几乎都会遇到 LoRA 或它的变体。

更重要的是，LoRA 提供了一种思维方式：

不要直接更新整个巨大对象。

先问：有效更新是否位于低维结构里？

如果是，就只学习那个结构。

这和现代 LLM 工程里的很多思想相通：sparse updates、adapter composition、quantized base + trainable delta、task-specific routing、model patching。

17. 常见误区

- 误区 1: LoRA 会让前向计算大幅减少。
 - 不准确。它主要减少可训练参数、梯度和 optimizer states；推理 merge 后结构不变。

- 误区 2: rank 越大一定越好。
 - 不一定。rank 是容量和正则的 trade-off。
- 误区 3: LoRA 是 adapter 的同义词。
 - 错。adapter 改 activation path, LoRA 改 weight update parameterization。
- 误区 4: LoRA 不会影响推理。
 - merge 后无额外结构延迟, 但不同 LoRA 权重当然会改变模型行为。
- 误区 5: 只要加 LoRA 就不用关心数据。
 - 错。LoRA 只是训练参数化方式, 数据质量仍然决定对齐方向。

18. 闭卷掌握检查

你应该能不看笔记回答:

1. LoRA 解决 full fine-tuning 的哪三个工程痛点?
2. 为什么 adapter 会引入 inference latency, 而 LoRA merge 后不会?
3. A、B、W0 的 shape 分别是什么?
4. 为什么 B=0, A=random 能保证初始不扰动?
5. alpha/r scaling 的作用是什么?
6. LoRA 参数量为什么是 $r(d_{in}+d_{out})$?
7. GPT-2 c_attn 上 LoRA 的参数量怎么手算?
8. LoRA 的 low intrinsic rank 假设是什么意思?
9. LoRA 实验证据必须同时看哪四个维度?
10. LoRA 的现代变体分别在改什么问题?

19. 用 AI agent 学这篇的正确方式

我正在读 LoRA。请你先让我画出 $W_0 x + BA x$ 的张量图。

然后问我 A/B/W0 的 shape, 要求我手算 $d=768, r=8$ 时的参数量。

接着让我解释为什么 B 零初始化、为什么 merge 后没有额外推理延迟。

最后请基于 `learning/lora-family/src/lora_minimal.py` 给我设计一个 merge 前后数值一致性测试。不要直接替我总结, 先考我。

真正掌握 LoRA 的标志是: 你能从 full fine-tuning 的 ΔW 出发, 解释为什么低秩分解可能够用, 能手写 forward 和 merge, 能判断 rank/alpha/target_modules 对训练和部署的影响。